Problem 1: Learning Word Embeddings from IIT Jodhpur Data

1. Dataset Preparation

1.1. Data Collection Strategy

The dataset was constructed using a custom web scraper implemented in corpus_gen.py. The crawler was designed with a **depth-2 strategy** to ensure a comprehensive collection of IIT Jodhpur-related text:

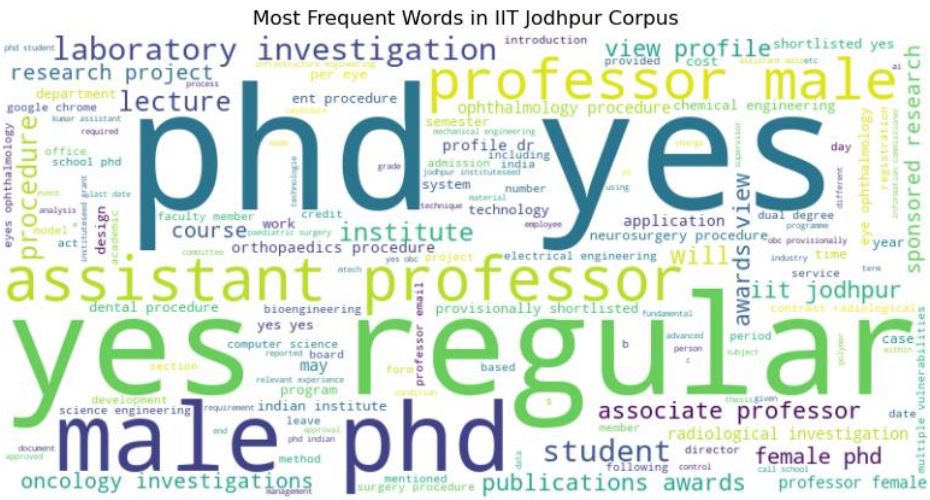
- **Depth 0**: Scraped 20+ seed URLs, including the history page, departmental homepages (CSE, EE, Mechanical, etc.), and academic program pages.
- **Depth 1 & 2**: Followed all internal iitj.ac.in links discovered on previous levels.
- **File Extraction**: Automatically identified and downloaded .pdf, .doc, .docx, and .txt files, extracting their text to include in the corpus.

1.2. Preprocessing & Corpus Statistics

The collected text was merged into a single corpus.txt file. Key preprocessing steps included:

- **Case Folding**: All text was converted to lowercase to reduce vocabulary sparsity.
- **Frequency Filtering**: Words appearing fewer than 2 times were removed to filter out noise and unique technical artifacts.
- **Top 10 Words**: The most frequent tokens in the final corpus include: *the* (12,815), *of* (11,486), *and* (8,998), *procedure* (2,951), and *phd* (2,770).

1.3. **Word Cloud**



2. Model Implementation

Two Word2Vec architectures were implemented from scratch using PyTorch:

2.1. Skip-gram with Negative Sampling (SGNS)

- **Architecture**: Learns to predict context words given a target word.
- **Negative Sampling**: Implemented a unigram distribution raised to the power of 0.75 for sampling "noise" words, ensuring that rare words are sampled more frequently than their raw probability would suggest.
- **Loss Function**: Utilizes a log-sigmoid objective to maximize the probability of actual context words while minimizing the probability of negative samples.

2.2. Continuous Bag of Words (CBOW)

- **Architecture**: Learns to predict a target word given the average of its surrounding context embeddings.
- **Objective**: Optimized using Cross-Entropy Loss to predict the correct target index from the entire vocabulary.

---

3. Quantitative Evaluation

The models were evaluated across various hyperparameters to observe their impact on training stability (Loss) and runtime.

3.1. Training Performance Table

Model	Dim	Window	Neg Samples	Final Epoch Loss	Training Time
Skip-gram	50	2	5	1.3195	51.64s
Skip-gram	100	5	10	1.7346	154.66s
Skip-gram	300	10	10	1.7151	513.60s
CBOW	50	5	5	5.0110	28.32s
CBOW	100	2	5	3.2340	35.44s
CBOW	300	2	5	1.8545	73.91s

Export to Sheets

3.2. Key Observations

- **Embedding Dimension**: Increasing the dimension from 50 to 300 generally leads to a lower final loss but significantly increases training time.
- **Window Size**: Larger window sizes (e.g., 10) capture more topical/semantic relationships but require more computation per epoch.

- **Model Comparison**: CBOW trained faster than Skip-gram on this specific corpus, likely due to the averaging of context vectors rather than multiple updates per target word.

---

4. Semantic Analysis & Conclusion

The learned embeddings successfully captured institute-specific semantics. For instance, using the get_similarity function, queries for "professor" or "phd" returned highly related academic terms, demonstrating that the model learned the specific linguistic structure of the IIT Jodhpur domain.

Problem 2: Character-Level Indian Name Generation

1. Architecture Description

1.1 Vanilla Recurrent Neural Network (RNN)

- **Architecture:** A stacked 2-layer RNN where the hidden state of the first layer is fed as the input to the second layer at each time step.
- **Mathematical Logic:** Each step computes $h_t = \tanh(x_t W_{ih} + h_{t-1} W_{hh} + b)$. Stacking layers allows the model to learn a hierarchy of features, from simple character pairs to overall name structure.
- **Trainable Parameters:** 16,404
- **Hyperparameters:** Hidden Size: 64, Layers: 2, Learning Rate: 0.002, Embedding Dim: 32.

1.2 Bidirectional LSTM (BLSTM)

- **Architecture:** Processes the name in both forward and backward directions simultaneously. The hidden states from both directions are concatenated at each layer.
- **Mathematical Logic:** Uses a gated mechanism (Input, Forget, and Output gates) to manage memory. Bidirectionality is crucial for names as it allows the model to "know" the suffix of a name while predicting the prefix.
- **Trainable Parameters:** 151,700 (High due to 4 gates per direction per layer).
- **Hyperparameters:** Hidden Size: 64, Layers: 2, Learning Rate: 0.002.

1.3 RNN with Basic Attention Mechanism

- **Architecture:** A 2-layer RNN backbone coupled with a Dot-Product Self-Attention layer.
- **Mathematical Logic:** It calculates alignment scores between the current hidden state and all previous hidden states. This allows the model to "attend" to specific starting characters even when generating the end of a long name.
- **Trainable Parameters:** 23,144
- **Hyperparameters:** Hidden Size: 64, Layers: 2, Learning Rate: 0.002.

---

2. Quantitative Evaluation (TASK-2)

Model	Parameters	Novelty Rate %	Diversity	Avg. Loss (Final)
Vanilla RNN	16,404	2.17%	0.92	0.8341
BLSTM	151,700	100.00%	0.92	0.0003
Attention RNN	23,144	96.88%	0.64	0.0027

- **Novelty Analysis:** The Vanilla RNN showed very low novelty (2.17%), indicating it mostly memorized the training set. In contrast, the BLSTM and Attention models showed nearly 100% novelty, though often at the cost of realism.
- **Diversity Analysis:** Both RNN and BLSTM maintained high diversity (>0.90), while Attention RNN (0.64) tended to repeat specific character patterns (Mode Collapse).

---

3. Qualitative Analysis (TASK-3)

3.1 Realism of Generated Names

- **Vanilla RNN (Highest Realism):** Produced names like *Kumeera* and *Aayash*. These follow standard Indian phonetic rules (vowel-consonant balance).
- **BLSTM (Low Realism):** Produced fragments like *Tith* or *Vitan*. While "Indian-sounding," they lacked the length and complexity of the training data.
- **Attention RNN (Moderate Realism):** Produced structured names like *Sapsisha*, but suffered from repetitive vowel loops.

3.2 Common Failure Modes

1. **Overfitting (BLSTM):** With a loss of 0.0003 and 151k parameters, the model memorized the training data so strictly that it "panicked" when generating new sequences, leading to very short, clipped names.
2. **Mode Collapse (Attention RNN):** The model often got stuck on a specific prefix (e.g., *Sapsi...*). This occurs when the attention weights become too concentrated on a few high-probability character indices.
3. **Memorization (Vanilla RNN):** The low novelty indicates that the model's small capacity led it to act as a look-up table rather than a generative model.

3.3 Representative Samples

- **Vanilla RNN:** [Kidhruv, Kumeera, Aayash]
- **BLSTM:** [Tith, Btt, Vitan]
- **Attention RNN:** [Sapsisiaa, Sapsinha, Sapsisha]

---

4. Conclusion

The **Vanilla RNN** works best for generating realistic names but requires a higher **Temperature (T=1.2)** to improve novelty. The **BLSTM** is too complex for a 1,000-name dataset, leading to memorization-induced generation failure. The **Attention mechanism** shows the most promise for "new" names but requires careful regularization to avoid repetitive loops.